```
"""
SELL & BUY Candidate Screener
-----------------------------
SELL side = forensic accounting red flags (Quantitative Value, ch. 3-4):
            Scaled total accruals (STA), scaled net operating assets (SNOA),
            Beneish probability of manipulation (PMAN) and a low Altman Z-score.
BUY side  = a DIFFERENT analysis (Quantitative Value, ch. 2 and 7):
            Gross profitability (GPA) for quality + a value ratio
            (book-to-market, EBIT/TEV or earnings yield) for price.

Data: universe.csv, built in Google Colab with yfinance (one row per company).
Every measure below is calculated in this file so the formulas are documented.
"""
import math

import numpy as np
import pandas as pd
import plotly.express as px
import streamlit as st

st.set_page_config(page_title="SELL & BUY Screener", page_icon="📊", layout="wide

# Rule 1 of the assignment: my 60-second pitch company can't be a recommendation,
# so it is dropped from the universe before anything is calculated.
PITCH_COMPANY = ["APP"]  # AppLovin

# ---------------------------------------------------------------------------
# YOUR WRITTEN CONCLUSIONS: replace the text below with your own analysis.
# ---------------------------------------------------------------------------
CONCLUSIONS = """
#### Top SELL candidates

**1. [Ticker - Company]**
*Why the screen flagged it:* [which red flags tripped and the actual numbers]
*Beyond the score:* [what you found in the 10-K, news, footnotes, cash flow trend
industry context... is the red flag real or explainable?]
*Conclusion:* [SELL / avoid, and what would change your mind]

**2. [Ticker - Company]**
...

#### Top BUY candidates

**1. [Ticker - Company]**
```

```
*Why the screen picked it:* [GPA, value ratio and percentiles]
*Beyond the score:* [competitive position, why it might be cheap, risks]
*Conclusion:* [BUY, and the key risk to watch]

**2. [Ticker - Company]**
...

#### Overall
[What the screen taught you, which settings you used, and any limitations.]
"""


# ---------------------------------------------------------------------- data ----
@st.cache_data
def load_data(path="universe.csv"):
    raw = pd.read_csv(path)
    raw = raw[~raw["ticker"].isin(PITCH_COMPANY)].reset_index(drop=True)
    return raw


def div(a, b):
    """Safe division: returns NaN (not an error or infinity) when data is missing
    a = pd.to_numeric(a, errors="coerce")
    b = pd.to_numeric(b, errors="coerce")
    out = a / b.where(b != 0)
    return out.replace([np.inf, -np.inf], np.nan)


def normal_cdf(x):
    return 0.5 * (1 + math.erf(x / math.sqrt(2))) if pd.notna(x) else np.nan


@st.cache_data
def compute_measures(raw):
    d = raw.copy()
    ta, ta_p = d["total_assets"], d["total_assets_prev"]
    sales, sales_p = d["revenue"], d["revenue_prev"]
    # Gross profit; if Yahoo doesn't report it, use revenue - cost of revenue
    gp = d["gross_profit"].fillna(sales - d["cost_of_revenue"])
    gp_p = d["gross_profit_prev"].fillna(sales_p - d["cost_of_revenue_prev"])

    # ---------------- SELL measures ----------------
    # STA = (net income - cash flow from operations) / total assets
    d["STA"] = div(d["net_income"] - d["operating_cash_flow"], ta)

    # SNOA = (operating assets - operating liabilities) / total assets
    #   operating assets      = total assets - cash
```

```
#   operating liabilities = total assets - total debt - minority interest - p
op_assets = ta - d["cash"].fillna(0)
op_liab = (ta - d["total_debt"].fillna(0) - d["minority_interest"].fillna(0)
           - d["preferred_stock"].fillna(0) - d["book_equity"])
d["SNOA"] = div(op_assets - op_liab, ta)

# Beneish M-score (8 variables) -> PMAN = probability of manipulation
c = pd.DataFrame(index=d.index)
c["DSRI"] = div(div(d["receivables"], sales), div(d["receivables_prev"], sale
c["GMI"] = div(div(gp_p, sales_p), div(gp, sales))
c["AQI"] = div(1 - div(d["current_assets"] + d["ppe"], ta),
               1 - div(d["current_assets_prev"] + d["ppe_prev"], ta_p))
c["SGI"] = div(sales, sales_p)
dep, dep_p = d["depreciation"], d["depreciation_prev"]
c["DEPI"] = div(div(dep_p, dep_p + d["ppe_prev"]), div(dep, dep + d["ppe"]))
c["SGAI"] = div(div(d["sga"], sales), div(d["sga_prev"], sales_p))
c["LVGI"] = div(div(d["current_liabilities"] + d["long_term_debt"].fillna(0),
                div(d["current_liabilities_prev"] + d["long_term_debt_prev"].
c["TATA"] = div(d["net_income"] - d["operating_cash_flow"], ta)
index_cols = ["DSRI", "GMI", "AQI", "SGI", "DEPI", "SGAI", "LVGI"]
n_imputed = c[index_cols].isna().sum(axis=1)
filled = c[index_cols].fillna(1.0)  # a missing index is set to 1.0 = "no cha
m = (-4.84 + 0.920 * filled["DSRI"] + 0.528 * filled["GMI"] + 0.404 * filled[
     + 0.892 * filled["SGI"] + 0.115 * filled["DEPI"] - 0.172 * filled["SGAI"
     + 4.679 * c["TATA"] - 0.327 * filled["LVGI"])
m = m.where((n_imputed <= 3) & c["TATA"].notna())  # too much missing -> no s
for col in index_cols + ["TATA"]:
    d["B_" + col] = c[col]
d["M_score"] = m
d["PMAN"] = m.apply(normal_cdf)
d["PMAN_imputed"] = n_imputed

# Altman Z = 1.2 WC/TA + 1.4 RE/TA + 3.3 EBIT/TA + 0.6 MVE/TL + 1.0 Sales/TA
d["Altman_Z"] = (1.2 * div(d["current_assets"] - d["current_liabilities"], ta
                 + 1.4 * div(d["retained_earnings"], ta)
                 + 3.3 * div(d["ebit"], ta)
                 + 0.6 * div(d["market_cap"], d["total_liabilities"])
                 + 1.0 * div(sales, ta))

# ---------------- BUY measures ----------------
d["GPA"] = div(gp, ta)  # gross profitability (Novy-Marx)
d["BM"] = div(d["book_equity"].where(d["book_equity"] > 0), d["market_cap"])
tev = d["market_cap"] + d["total_debt"].fillna(0) - d["cash"].fillna(0)
d["EBIT_TEV"] = div(d["ebit"], tev.where(tev > 0))
d["EY"] = div(d["net_income"], d["market_cap"])  # earnings yield = 1 / (P/E)

d["Mkt_cap_B"] = d["market_cap"] / 1e9
```

```
    return d


raw = load_data()
data = compute_measures(raw)

LABELS = {
    "STA": "STA", "SNOA": "SNOA", "PMAN": "PMAN (Beneish)", "Altman_Z": "Altman Z
    "GPA": "GPA", "BM": "Book-to-market", "EBIT_TEV": "EBIT / TEV", "EY": "Earnin
    "Mkt_cap_B": "Market cap ($B)",
}
VALUE_MEASURES = {"Book-to-market (B/M)": "BM", "EBIT / TEV (enterprise multiple)
                  "Earnings yield (E/P = 1 / P/E)": "EY"}

# ------------------------------------------------------------------ sidebar ----
with st.sidebar:
    st.header("Universe filters")
    all_sectors = sorted(data["sector"].dropna().unique())
    sectors = st.multiselect("Sectors to include", all_sectors, default=all_secto
    min_cap = st.slider("Minimum market cap ($B)", 0.0, 50.0, 0.0, 0.5)
    excl_fin = st.checkbox("Exclude Financials (banks/insurers)", value=True,
                           help="Accrual, SNOA and Altman Z measures were not des
    within_sector = st.checkbox("Rank within each sector", value=False,
                                help="Percentiles compare each company only with

    st.header("SELL rule: forensic red flags")
    sta_cut = st.slider("STA: flag the worst __% (highest accruals)", 1, 50, 10)
    snoa_cut = st.slider("SNOA: flag the worst __% (most bloated)", 1, 50, 10)
    pman_cut = st.slider("PMAN: flag the worst __% (most likely manipulators)", 1
    z_cut = st.slider("Altman Z: flag below", 0.0, 3.0, 1.81, 0.01,
                      help="1.81 is Altman's distress-zone cutoff.")
    n_flags = st.slider("Red flags required to be a SELL", 1, 4, 2)

    st.header("BUY rule: quality + value")
    value_label = st.selectbox("Value measure", list(VALUE_MEASURES))
    q_weight = st.slider("Weight on quality (GPA), %", 0, 100, 50)
    buy_cut = st.slider("Minimum BUY score (percentile)", 50, 99, 85)
    max_buy_flags = st.slider("Max red flags a BUY may have", 0, 3, 0)

value_col = VALUE_MEASURES[value_label]

# ------------------------------------------------------------------ screening ----
u = data[data["sector"].isin(sectors) & (data["Mkt_cap_B"].fillna(0) >= min_cap)]
if excl_fin:
    u = u[u["sector"] != "Financials"]
```

```
def pct_rank(s):
    """Percentile 0-100 within the screened universe (higher value = higher perce
    if within_sector:
        return s.groupby(u["sector"]).rank(pct=True) * 100
    return s.rank(pct=True) * 100


for col in ["STA", "SNOA", "PMAN", "GPA", value_col]:
    u[col + "_pct"] = pct_rank(u[col])

u["flag_STA"] = u["STA_pct"] >= 100 - sta_cut
u["flag_SNOA"] = u["SNOA_pct"] >= 100 - snoa_cut
u["flag_PMAN"] = u["PMAN_pct"] >= 100 - pman_cut
u["flag_Z"] = u["Altman_Z"] < z_cut  # NaN compares False: missing data never cre
flag_cols = {"flag_STA": "STA", "flag_SNOA": "SNOA", "flag_PMAN": "PMAN", "flag_Z
u["Red_flags"] = u[list(flag_cols)].sum(axis=1)
u["Which_flags"] = u.apply(lambda r: ", ".join(v for k, v in flag_cols.items() if
u["SELL"] = u["Red_flags"] >= n_flags

w = q_weight / 100
u["BUY_score"] = w * u["GPA_pct"] + (1 - w) * u[value_col + "_pct"]
u["BUY"] = (u["BUY_score"] >= buy_cut) & (u["Red_flags"] <= max_buy_flags) & ~u["
u["Status"] = np.select([u["SELL"], u["BUY"]], ["SELL", "BUY"], "Neither")

sell = u[u["SELL"]].sort_values(["Red_flags", "STA_pct"], ascending=False)
buy = u[u["BUY"]].sort_values("BUY_score", ascending=False)

# ---------------------------------------------------------------- page ----
st.title("📊 SELL & BUY Candidate Screener")
pulled = raw["data_date"].iloc[0] if "data_date" in raw.columns else "n/a"
st.caption(f"Universe: {len(data):,} US public companies (S&P indexes) · data pul
           f"{', '.join(PITCH_COMPANY)} excluded (my pitch company). "
           "SELL = forensic accounting red flags. BUY = gross profitability + val

m1, m2, m3 = st.columns(3)
m1.metric("Companies screened", f"{len(u):,}")
m2.metric("SELL candidates", f"{len(sell):,}")
m3.metric("BUY candidates", f"{len(buy):,}")

NUM = st.column_config.NumberColumn
left, right = st.columns(2)
with left:
    st.subheader("🔻 SELL candidates")
    st.caption(f"At least {n_flags} red flag(s) out of: STA, SNOA, PMAN, Altman Z
    t = sell[["ticker", "name", "sector", "Red_flags", "Which_flags", "STA", "STA
              "SNOA_pct", "PMAN", "Altman_Z"]]
    st.dataframe(t, hide_index=True, height=420, column_config={
```

```
        "ticker": "Ticker", "name": "Company", "sector": "Sector", "Red_flags": "
        "Which_flags": "Which flags", "STA": NUM("STA", format="%.3f"), "STA_pct"
        "SNOA": NUM("SNOA", format="%.2f"), "SNOA_pct": NUM("SNOA pct", format="%
        "PMAN": NUM("PMAN", format="%.2f"), "Altman_Z": NUM("Altman Z", format="%
    st.download_button("Download SELL list (CSV)", t.to_csv(index=False), "sell_c
with right:
    st.subheader("🔷 BUY candidates")
    st.caption(f"BUY score = {q_weight}% GPA percentile + {100 - q_weight}% {valu
               f"percentile ≥ {buy_cut}, with at most {max_buy_flags} red flag(s)
    t = buy[["ticker", "name", "sector", "BUY_score", "GPA", "GPA_pct", value_col
             "Mkt_cap_B", "Red_flags"]]
    st.dataframe(t, hide_index=True, height=420, column_config={
        "ticker": "Ticker", "name": "Company", "sector": "Sector", "BUY_score": N
        "GPA": NUM("GPA", format="%.3f"), "GPA_pct": NUM("GPA pct", format="%.0f"
        value_col: NUM(LABELS[value_col], format="%.3f"), value_col + "_pct": NUM
        "Mkt_cap_B": NUM("Mkt cap $B", format="%.1f"), "Red_flags": "Flags"})
    st.download_button("Download BUY list (CSV)", t.to_csv(index=False), "buy_can

tab_chart, tab_drill, tab_concl, tab_method, tab_data = st.tabs(
    ["Chart", "Company drill-down", "Conclusions", "Methodology", "Data coverage"

with tab_chart:
    opts = list(LABELS)
    c1, c2 = st.columns(2)
    x = c1.selectbox("X axis", opts, index=opts.index("STA"), format_func=LABELS.
    y = c2.selectbox("Y axis", opts, index=opts.index("GPA"), format_func=LABELS.
    p = u.dropna(subset=[x, y])
    for col in {x, y}:  # trim the most extreme 1% on each side so the chart stay
        lo, hi = p[col].quantile([0.01, 0.99])
        p = p[p[col].between(lo, hi)]
    if p.empty:
        st.info("No companies have data for both measures under the current filte
    else:
        fig = px.scatter(p, x=x, y=y, color="Status", hover_name="ticker",
                         hover_data={"name": True, "Which_flags": True, "Status":
                         labels={x: LABELS[x], y: LABELS[y]}, height=560, opacity
                         category_orders={"Status": ["SELL", "BUY", "Neither"]},
                         color_discrete_map={"SELL": "#D35400", "BUY": "#1F6FB2",
        st.plotly_chart(fig)
        st.caption(f"{len(p):,} companies plotted. Move any sidebar slider and th

with tab_drill:
    choices = u.sort_values("ticker")
    pick = st.selectbox("Company", choices["ticker"], index=None, placeholder="Ty
                        format_func=lambda t: f"{t} - {choices.set_index('ticker'
    if pick:
        r = u.set_index("ticker").loc[pick]
```

```
badge = {"SELL": "🔻 SELL candidate", "BUY": "🔷 BUY candidate", "Neither
st.markdown(f"### {r['name']} ({pick}): {badge}")
st.caption(f"{r['sector']} / {r['industry']} · market cap ${r['Mkt_cap_B'
           f"fiscal year end {r.get('fiscal_year_end', 'n/a')}")

def fmt(v, d=3):
    return "n/a (missing data)" if pd.isna(v) else f"{v:.{d}f}"

def pctf(v):
    return "n/a" if pd.isna(v) else f"{v:.0f}"

rows = [
    ["STA (accruals)", fmt(r["STA"]), pctf(r["STA_pct"]), "🚩" if r["flag
    ["SNOA", fmt(r["SNOA"]), pctf(r["SNOA_pct"]), "🚩" if r["flag_SNOA"]
    ["PMAN (Beneish)", fmt(r["PMAN"]), pctf(r["PMAN_pct"]), "🚩" if r["fl
    ["Altman Z", fmt(r["Altman_Z"], 2), "", "🚩" if r["flag_Z"] else "",
    ["GPA", fmt(r["GPA"]), pctf(r["GPA_pct"]), "", "Gross profit / TA"],
    [LABELS[value_col], fmt(r[value_col]), pctf(r[value_col + "_pct"]), "
    ["BUY score", pctf(r["BUY_score"]), "", "", f"{q_weight}% GPA pct + {
]
st.dataframe(pd.DataFrame(rows, columns=["Measure", "Value", "Percentile"
             hide_index=True)
st.caption("Percentiles run 0-100 within the screened universe. For STA,
           "percentile is bad; for GPA and the value measure a HIGH perce

notes = []
if pd.notna(r["STA"]):
    notes.append("Earnings are **backed by cash** (STA ≤ 0)." if r["STA"]
                 "Reported earnings are **running ahead of operating cash
if pd.notna(r["M_score"]):
    notes.append(f"Beneish M-score of {r['M_score']:.2f} is "
                 + ("**above** the classic -1.78 cutoff (likely manipulat
                    if r["M_score"] > -1.78 else "below the classic -1.78
                 + (f" ({int(r['PMAN_imputed'])} of the 7 Beneish indexes
                    if r["PMAN_imputed"] else ""))
if pd.notna(r["Altman_Z"]):
    zone = "safe" if r["Altman_Z"] > 2.99 else "grey" if r["Altman_Z"] >=
    notes.append(f"Altman Z puts it in the **{zone} zone**.")
notes.append(f"Red flags tripped: **{r['Which_flags'] or 'none'}**.")
st.markdown("\n".join("- " + n for n in notes))

with st.expander("Beneish M-score components"):
    comp = {k: r["B_" + k] for k in ["DSRI", "GMI", "AQI", "SGI", "DEPI",
    st.dataframe(pd.DataFrame({"Index": comp.keys(), "Value": [fmt(v) for
                 hide_index=True)
    st.caption("Values above 1 on DSRI, GMI, AQI, SGI, DEPI and SGAI push
               "Missing indexes are set to 1.0 (no change).")
```

```
    else:
        st.info("Pick a company to see every measure, its percentile and which fl

with tab_concl:
    st.markdown("##### Current top names under your filter settings")
    c1, c2 = st.columns(2)
    c1.markdown("**Top SELL:** " + (", ".join(f"{t} ({f} flags)" for t, f in
                                            zip(sell["ticker"].head(5), sell["
    c2.markdown("**Top BUY:** " + (", ".join(f"{t} ({s:.0f})" for t, s in
                                           zip(buy["ticker"].head(5), buy["BUY
    st.divider()
    st.markdown(CONCLUSIONS)

with tab_method:
    st.markdown(f"""
**Universe.** US companies in the S&P 500 and S&P MidCap 400 (and SmallCap 600 if
build), pulled from Wikipedia and Yahoo Finance with `yfinance`. Latest annual fi
cap = latest price × shares outstanding. {', '.join(PITCH_COMPANY)} (my 60-second
Financials are excluded by default because accrual, SNOA and Z-score models were

**SELL side: is the accounting trustworthy and is the firm healthy?** (Quantitati

| Measure | Formula | Red flag when |
|---|---|---|
| STA | (Net income − Cash flow from operations) / Total assets | in the worst {s
| SNOA | (Operating assets − Operating liabilities) / Total assets, where operati
| PMAN | Φ(M), M = −4.84 + 0.920 DSRI + 0.528 GMI + 0.404 AQI + 0.892 SGI + 0.115
| Altman Z | 1.2 WC/TA + 1.4 RE/TA + 3.3 EBIT/TA + 0.6 MVE/TL + 1.0 Sales/TA | be

A company is a **SELL** candidate with at least **{n_flags}** red flag(s). Requir
reduces false alarms from one odd number.

**BUY side: a different analysis, quality at a good price** (Quantitative Value c

| Measure | Formula | Better when |
|---|---|---|
| GPA | Gross profit / Total assets | higher (more profitable assets) |
| Book-to-market | Book equity / Market cap (negative equity excluded) | higher (
| EBIT / TEV | EBIT / (Market cap + Total debt − Cash) | higher (cheaper) |
| Earnings yield | Net income / Market cap (= 1 / P/E) | higher (cheaper) |

BUY score = {q_weight}% × GPA percentile + {100 - q_weight}% × {value_label.split
A **BUY** candidate scores at least **{buy_cut}**, has at most **{max_buy_flags}*
a SELL. The BUY rule is not the SELL rule flipped: it uses profitability and pric

**Missing data.** Every ratio uses safe division: missing or zero denominators gi
number. Blank measures are left out of percentile ranks and can never trigger a r
```

```
Beneish index is set to 1.0 (no change), and if more than 3 of the 7 indexes are
are missing, no PMAN is calculated. See the *Data coverage* tab for how many comp

**Limitations.** One year of statements; Yahoo data can contain gaps or classific
screens point to where to dig further and are not recommendations on their own.
""")

with tab_data:
    cov = pd.DataFrame({
        "Measure": [LABELS[c] for c in ["STA", "SNOA", "PMAN", "Altman_Z", "GPA",
        "Companies with data": [int(u[c].notna().sum()) for c in ["STA", "SNOA",
    })
    cov["Missing"] = len(u) - cov["Companies with data"]
    cov["Coverage %"] = (cov["Companies with data"] / max(len(u), 1) * 100).round
    st.dataframe(cov, hide_index=True)
    st.caption(f"Out of {len(u):,} companies in the current screen. Missing value
               "are never counted as red flags.")
```